

AI Report

We predict this text is

Human Generated

AI Probability

12%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

INTERNATIONAL ISLAMIC UNIVERSITY - 2/11/2026

Turja Dutta

INTERNATIONAL ISLAMIC UNIVERSITY

CHITTAGONG

CacheFlow: Memory-Aware Expert Scheduling

for Efficient Mixture-of-Experts Models

Supervised by:

Jamil As Ad

Lecturer

Department of Computer Science and Engineering

International Islamic University Chittagong

Submitted By:

Mir Md. Tarhimul Quader C221017

Md. Ifthekar Ahamed Sayem C221020

Turja Dutta C221026

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

AND ENGINEERING

Department of Computer Science and Engineering

International Islamic University Chittagong

February 2026

Declaration

We hereby affirm the following statements regarding our thesis:

The thesis has been successfully completed as part of our undergraduate degree program at International Islamic University Chittagong.

The thesis work does not contain any previously published or third-party content without proper citation.

The thesis work has not been previously submitted for any other degree or diploma at any other university or institution.

We have appropriately acknowledged all significant sources of contribution in the thesis.

Student's Full Name and Metric ID:

Mir Md. Tarhimul Quader (C221017)

Md. Iftaker Ahamed Sayem (C221020)

Turja Dutta (C221026)

Supervisor's Declaration

I formally state that I have examined this thesis and claim it to be of sufficient quality and scope to be granted the undergraduate degree of Bachelor of Science in Computer Science and Engineering.

Jamil As-Ad

Lecturer

Department of Computer Science and Engineering

International Islamic University Chittagong

Mohammed Arif Asfe

Lecturer

Department of Computer Science and Engineering

Southern University Chittagong

Dedication

This thesis report is dedicated to us, our supervisor, and our family. The teamwork was satisfactory and the family's support was incredibly Amazing.

Our dedicated and most hard-working Supervisor and co-supervisor, who has been a constant support throughout these months. In this document, the contributions are acknowledged too.

Acknowledgment

To start with, All the praises to the Almighty Allah, for his mercy because of which we were able to finish our thesis despite having so many obstacles.

Secondly, we would like to extend our gratitude to our supervisor, Jamil As Ad Sir, for his continuous effort and guidelines from the very beginning of our research.

Ethical Statement

Hereby we state that None of the unethical practices were used in the completion of our thesis work. The data we used for the research purpose are original. We carefully checked every citation we used here. The writers of the work accept all the liabilities for any kind of violation of the thesis rule.

Abstract

Mixture-of-Experts (MoE) architectures provide the ability to provide large language models at reasonable scale, in that only a small number of expert networks are executed per-token. This is though sparseness reduces compute costs, countering inference time deployment since most systems retain all experts in a memory on a single, high-performance, device, which demand high memory bandwidth and cause out-of-memory errors on a standard computer. This thesis presents the priority based scheduler CacheFlow that renders MoE inference memory-efficient. CacheFlow is a Python middleware, which is connected to the forward pass of a rebuilt Qwen2-MoE model, which provides a direct control of who the experts execute and which ones remain in memory. It sees the GPU memory as a small expert cache and employs the lightweight priority function that takes into account the prevalence of context reuse, proximity of experts. Experiments using real inference and quantized weights demonstrate that consistent results may be achieved even using a small number of working experts. By reducing in-memory expert count, ease of 60 experts down to 16, CacheFlow reduces the active expert memory by approximately 73% and reduces the total amounts of gpu memory utilization by 47-50%, including the static parts of the model, which remain in memory. Consequently, even large MoE models of about 14 billion parameters can be trained on hardware where standard inference algorithms do not perform. Other works show that CacheFlow retains equitable expert delivery, holds high temporal locality in expert utilization, and is significantly memory efficient, employing about 7,132.5 MB less with the use of GPUs in inference. These results demonstrate that the use of memory-conscious scheduling is a successful method of resolving the theoretical aspects of scaling of MoE into practice and practical memory-intensive deployments.

Keywords: CacheFlow, Expert Scheduling, GPU Memory Management, Large Language Models, Memory-Constrained Inference, Mixture-of-Experts, MoE Inference.

Contents

Declaration i

Supervisor's Declaration ii

Dedication iii

Acknowledgment iv

Ethical Statement v

Abstract vi

Table of Contents x

List of Figures xi

List of Tables xii

1 Introduction 1

1.1 Research Background 1

1.2 Problem Statement 2

1.3 Motivation of the Research 3

1.4 Objectives 4

1.5 Organization of the Thesis 5

1.6 Summary 6

2 Literature Review 7

2.1 Overview 7

2.2 Foundational and Modern Architectures for Scalable Neural Models 8

2.2.1 Adaptive Mixtures of Local Experts by Jacobs et al. (1991) 8

2.2.2 Hierarchical Mixtures of Experts by Jordan and Jacobs (1994) 9

2.2.3 Adam: A Method for Stochastic Optimization by Kingma and Ba (2014) 9

vii

2.2.4 Distilling the Knowledge in a Neural Network by Hinton et al. (2015) 9

2.2.5 Neural Machine Translation by Jointly Learning to Align and Translate by Bahdanau et al. (2015) 10

2.2.6 Batch Normalization by Ioffe and Szegedy (2015) 10

2.2.7 Layer Normalization by Ba et al. (2016) 10

2.2.8 Gaussian Error Linear Units (GELU) by Hendrycks and Gimpel (2016) 11

2.2.9 Attention Is All You Need by Vaswani et al. (2017) 11

2.2.10 Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer by Shazeer et al. (2017) 11

2.2.11 Efficient Softmax Approximation for GPUs by Grave et al. (2017) 12

2.2.12 Improving Language Understanding by Generative Pre-Training by Radford et al. (2018) 12

2.2.13 Breaking the Softmax Bottleneck by Yang et al. (2018) 12

2.2.14 Language Models are Unsupervised Multitask Learners by Radford et al. (2019)	13
2.2.15 Generating Long Sequences with Sparse Transformers by Child et al. (2019)	13
2.2.16 Root Mean Square Layer Normalization by Zhang and Sennrich (2019)	13
2.2.17 Language Models are Few-Shot Learners by Brown et al. (2020)	14
2.2.18 Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer by Raffel et al. (2020)	14
2.2.19 DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters by Rasley et al. (2020)	14
2.2.20 An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale by Dosovitskiy et al. (2020)	15
2.2.21 GLU Variants Improve Transformer by Shazeer (2020)	15
2.2.22 Longformer: The Long-Document Transformer by Beltagy et al. (2020)	15
2.2.23 ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations by Lan et al. (2020)	16
2.2.24 GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding by Lepikhin et al. (2021)	16
2.2.25 Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity by Fedus et al. (2022)	16
2.2.26 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness by Dao et al. (2022)	17
2.2.27 FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning by Dao (2023)	17
2.2.28 Efficient Memory Management for Large Language Model Serving with PagedAttention by Kwon et al. (2023)	17
2.2.29 LLaMA: Open and Efficient Foundation Language Models by Touvron et al. (2023)	18
2.2.30 SwapMoE: Serving Off-the-Shelf MoE-based Large Language Models with Tunable Memory Budget by Yoo and Lee (2023)	18
2.2.31 MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache by Zuo et al. (2023)	19
2.2.32 DeepSpeed-MII: Model Inference Interface by Microsoft (2023)	19
2.2.33 TensorRT-LLM: Optimized Inference for Large Language Models by NVIDIA (2023)	19
2.2.34 DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model by Liang et al. (2024)	20
2.2.35 fMoE: Fine-Grained and Prompt-Aware Expert Offloading for Large Mixture-of-Experts Serving by Shen et al. (2024)	20
2.3 Comparative Summary of Reviewed Systems	20

2.4 Research Gap	23
2.5 Summary	23
3 Methodology	25
3.1 System Overview	25
3.2 The CacheFlow Scheduling Algorithm	27
3.2.1 Priority Scoring Mechanism	27
3.2.2 Request Queue and Ordering	27
3.2.3 Virtual LRU and Expert Residency Tracking	28
3.2.4 Eviction Policy and Capacity Constraint	28
3.3 Model Implementation (Custom Qwen2-MoE)	29
3.3.1 Manual Reconstruction of the MoE Architecture	29
3.3.2 Sparse Expert Layer: Qwen2MoeMLP	29
3.3.3 Gating Network and Top-K Routing	30
3.3.4 CacheFlow Integration via CacheFlowQwenBlock	30
3.3.5 Rationale for Manual Implementation	31
3.4 Optimization and Constraints	31
3.4.1 Memory Constraints and Design Assumptions	31
3.4.2 4-bit Quantization of Static Weights	32
3.4.3 Expert Capacity and Residency Limit	32
3.4.4 Execution Correctness Under Constraints	32
3.5 Experimental Environment	33
3.5.1 Hardware Configuration	33
3.5.2 Software Stack	33
3.5.3 Workload and Prompt Design	33
3.5.4 Experimental Protocol	34
3.6 Summary	34
4 Results and Discussions	36
4.1 Performance Analysis: The Capacity Trade-off	36

4.2 Behavioral Explainability	38
4.2.1 Expert Residency and Temporal locality	38
4.2.2 Expert Utilization and Fairness	39
4.2.3 Interpretability of Scheduling Decisions	40
4.3 Impact of Prompt Locality	40
4.4 Memory Optimization Analysis	42
4.4.1 Reduction in Active Expert Memory	42
4.4.2 Total GPU Memory Reduction	42
4.4.3 Practical Implications	43
4.5 Discussion	43
4.6 Summary	44
5 Conclusion	46
5.1 Research Summary	46
5.2 Contribution of the Work	47
5.3 Future Work	47
References	

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.